

Comparing Convolutional Neural Network Architectures for Image Classification on MNIST and CIFAR-10, with a Channel-Attention Residual Variant

Srinidhi Narla

Department of Computer Science
The University of Texas at Dallas
Student ID: SXN220141
sxn220141@utdallas.edu

Pranav Cheedalla

Department of Computer Science
The University of Texas at Dallas
Student ID: PXC230033
pxc230033@utdallas.edu

Abstract—We implement and compare seven neural network architectures for image classification on MNIST and CIFAR-10: a multilayer perceptron, a small custom CNN, LeNet-5, AlexNet, VGG-11, ResNet-18, and a novel SE-ResNet-Lite that augments a ResNet-18 backbone with Squeeze-and-Excitation channel attention and a depthwise-separable stem. All models are trained from scratch under a single data pipeline, optimizer schedule, and evaluation protocol, and compared on accuracy, macro precision, recall, F1, and macro one-vs-rest AUROC. SE-ResNet-Lite matches or exceeds the ResNet-18 baseline on both datasets while adding under 1% to its parameter count, demonstrating that lightweight channel recalibration gives a small but consistent gain on small-image classification. We also examine training dynamics through loss curves, class confusion structure through normalized confusion matrices, and penultimate-layer geometry through t-SNE projections, finding that the residual errors of every model are concentrated on the same small set of visually similar classes (cat/dog on CIFAR-10; 4/9 on MNIST).

Index Terms—image classification, convolutional neural networks, residual learning, squeeze-and-excitation, MNIST, CIFAR-10

I. Introduction (A: 50%, B: 50%)

Image classification is one of the canonical benchmarks for supervised deep-learning models. Even on small benchmark datasets such as MNIST [1] and CIFAR-10 [2], architectural choices have measurable effects on accuracy, parameter efficiency, and calibration. The space of credible architectures is also unusually broad: it spans flat fully connected networks, a half-century of convolutional designs (LeNet through ResNet), and a more recent family of designs that target either parameter efficiency (MobileNet) or learned feature recalibration (Squeeze-and-Excitation). For most modern projects, the question is not whether to use a CNN, but which CNN, and whether a small targeted modification of a strong baseline can be justified.

This project asks two concrete questions:

- 1) How do classical and modern CNN architectures compare on MNIST and CIFAR-10 when trained under a single fixed pipeline?
- 2) Can a small, low-cost modification (adding channel attention and a depthwise-separable stem to a ResNet-18 backbone) give measurable improvements without significantly increasing parameter count?

To answer these we re-implement seven models from scratch in PyTorch [3], train each on both datasets with an identical training schedule, and evaluate them with a consistent multi-metric protocol. The architectures span four decades of design practice: a flat multilayer perceptron, a small two-block CNN, the original LeNet-5 [1], AlexNet [4] adapted for 32×32 inputs, VGG-11 [5], ResNet-18 [6], and our own SE-ResNet-Lite, which combines residual learning with Squeeze-and-Excitation [7] channel attention and a depthwise-separable stem inspired by MobileNet [8]. The same training recipe is used across every model: 90/10 train/validation split, cosine learning rate annealing, dataset-appropriate augmentation, and best-validation checkpoint selection.

Why this novel architecture? ResNet-18 is the strongest classical baseline for CIFAR-10 and a natural starting point for a controlled architectural intervention. Squeeze-and-Excitation channel attention is one of the smallest, best-understood post-residual modifications in the literature, and depthwise-separable convolution is a well-established efficiency trick. Combining the two on a single backbone lets us ask the targeted empirical question: at a fixed parameter budget, does adding learned per-channel recalibration improve classification on small-image benchmarks?

Contributions.

- A reproducible PyTorch implementation of all seven models trained under identical conditions, including a complete training pipeline, evaluation harness, and

figure-generation scripts.

- A direct head-to-head comparison on MNIST and CIFAR-10 using five metrics (accuracy, macro precision/recall/F1, and macro one-vs-rest AUROC) along with confusion matrices and t-SNE feature visualizations.
- SE-ResNet-Lite, our novel architecture that combines SE blocks with a depthwise-separable stem, evaluated head-to-head against the same ResNet-18 baseline at less than 1% parameter overhead.
- An explicit discussion of the practical engineering choices that affect reproducibility on a small-laptop training budget, including a per-model-family optimizer rationale and a description of the Apple Silicon (MPS) backend caveats encountered during training.

The remainder of the paper is organized as follows. Section II surveys the relevant literature. Section III describes every implemented architecture and the optimization recipe. Section IV presents the empirical settings and the experimental results. Section V discusses the patterns we observe and the limitations of the study. Section VI concludes.

II. Related Work (A: 50%, B: 50%)

A. Classical convolutional networks (A: 50%, B: 50%)

LeNet-5 [1] established the convolution–pooling–MLP template that defines the modern CNN. It was originally designed for handwritten digit recognition (essentially the task posed by MNIST) and remains a strong baseline on that dataset even today. AlexNet [4] showed that significantly deeper and wider CNNs trained on GPUs could dramatically outperform classical methods on ImageNet, popularizing the combination of ReLU activations, dropout regularization, and large-scale data augmentation that has remained the workhorse recipe ever since. VGG [5] demonstrated that very deep networks built entirely from stacks of small 3×3 convolutions could yield further gains, at the cost of substantially more parameters than AlexNet. In their pure form these networks rely on careful initialization and small learning rates; modern implementations almost always add BatchNorm [9] after every convolution, which is the configuration we use here.

B. Residual learning (A: 50%, B: 50%)

ResNet [6] introduced identity-mapping skip connections that make training very deep networks tractable by reformulating each block as $\mathbf{y} = \mathbf{x} + \mathcal{F}(\mathbf{x})$, so that the network’s optimization target is the residual $\mathcal{F}(\mathbf{x})$ rather than the full transformation $\mathbf{y}$. The empirical consequence is that a 50-, 101-, or 152-layer network can be trained without the degradation that affects equally deep plain networks. ResNet-18, the smallest variant, has become the standard reference architecture for CIFAR-10 and small-image classification more generally. Modern CIFAR-style training recipes typically replace the original 7×7 stride-2 ImageNet stem with a single 3×3 convolution (the “CIFAR

stem”), use four residual stages with progressive down-sampling, and pair the network with SGD plus momentum and a cosine learning rate schedule.

C. Channel attention and the SE block (A: 50%, B: 50%)

Squeeze-and-Excitation [7] introduces a lightweight, parameter-efficient channel-attention mechanism. Given a feature map $\mathbf{F} \in \mathbb{R}^{C\times H\times W}$, the SE module squeezes $\mathbf{F}$ to a per-channel descriptor via global average pooling, excites the descriptor through a two-layer MLP with reduction ratio r and a sigmoid output, and uses the resulting per-channel scalar gate to rescale the original feature map. SE-ResNet, SE-Inception, and SE-VGG variants reliably yield small accuracy improvements at very small parameter overhead, and SE blocks have since been incorporated into many production architectures (EfficientNet, MobileNet-v3, ConvNeXt-v2, and others).

D. Parameter-efficient convolution (A: 50%, B: 50%)

MobileNet [8] popularized depthwise-separable convolution as a way to reduce both parameter count and FLOPs. A standard convolution of $k\times k$ kernel with C_{in} input channels and C_{out} output channels has $k^2 C_{\text{in}} C_{\text{out}}$ parameters. The factored form replaces it with (a) a depthwise convolution of $k\times k$ kernel grouped by input channel ($k^2 C_{\text{in}}$ parameters) followed by (b) a 1×1 pointwise convolution that mixes channels ($C_{\text{in}} C_{\text{out}}$ parameters), for a total of $C_{\text{in}}(k^2 + C_{\text{out}})$ parameters, roughly $1/k^2$ of the standard form when $C_{\text{out}} \gg 1$. The operation has the same receptive field as the standard convolution but a substantially smaller parameter and FLOP budget.

E. Position of this work (A: 50%, B: 50%)

Most prior work cited above focuses on either an isolated baseline or a single proposed modification reported on a single, very large dataset (typically ImageNet). For a course project conducted on a single laptop and two small benchmarks, what is most useful is a controlled, like-for-like comparison: every model trained on the same data, with the same augmentations, optimizer family, learning rate schedule, and evaluation protocol. We therefore re-train seven different architectures under a single pipeline and report the same five metrics for each, so that the effect of architectural depth, residual learning, and channel attention can be isolated from the effects of differing training recipes.

III. Proposed Approach (A: 50%, B: 50%)

All models are implemented in PyTorch and share the same input adapter, optimizer logic, learning-rate scheduler, data augmentation, and evaluation protocol; only the network body differs. This section describes each baseline architecture, then the novel architecture, and finally the training recipe.

A. Baseline architectures (A: 50%, B: 50%)

MLP. A 3-layer fully connected network with hidden sizes 512 and 256, ReLU activations, and dropout $p = 0.2$ between hidden layers. Inputs are flattened to a single vector. The model has no spatial structure and serves as a sanity baseline: any gain from a convolutional model should be attributed to the spatial inductive bias introduced by convolutions and pooling.

SimpleCNN. Two convolutional blocks (3×3 conv $\rightarrow$ ReLU $\rightarrow 2 \times 2$ max-pool) with channels 32 and 64, followed by a single 128-unit fully connected layer with dropout $p = 0.25$. A small, deliberately shallow CNN that establishes how much benefit comes from the basic convolution-pooling pattern alone.

LeNet-5 [1], [10]. The classic two-convolution, three-FC architecture with average pooling, in the configuration originally described for MNIST. MNIST inputs are zero-padded from 28×28 to 32×32 to match the original input size. For CIFAR-10 we keep the original $3 \rightarrow 6 \rightarrow 16$ channel progression and the $120 \rightarrow 84 \rightarrow 10$ classifier head.

AlexNet (small) [4]. The classical five-conv plus three-FC stack, adapted to small inputs by reducing the initial kernel size from 11×11 to 3×3 , removing the original stride, and inserting an adaptive average pool before the classifier so the same network operates on both 28×28 and 32×32 inputs. A 1×1 adapter convolution maps grayscale inputs to three channels so the network is shared across datasets. The classifier uses two 1024/512 hidden layers with dropout $p = 0.5$. The model has no BatchNorm, which is why the training recipe described in Section III-C uses Adam rather than the high-LR SGD recipe used for the BatchNorm-based deeper models.

VGG-11 [5]. The “configuration A” VGG with eight convolutional layers (channel widths 64, 128, 256, 256, 512, 512, 512, 512), five max-poolings, and BatchNorm [9] after every convolution. An adaptive 1×1 average pool feeds a small two-layer FC head ($512 \rightarrow 512 \rightarrow \text{num_classes}$) with dropout $p = 0.5$. Because five stride-2 max-poolings would collapse the 28×28 MNIST input to a sub-unit spatial size, we enable `ceil_mode=True` on each pool so MNIST training does not crash.

ResNet-18 [6]. Eight basic residual blocks organized into four stages of widths 64, 128, 256, 512 with strided down-sampling between stages and BatchNorm after every convolution. We use the CIFAR-style stem (a single 3×3 convolution) rather than the original ImageNet 7×7 stride-2 stem, since the latter destroys too much spatial information on 32×32 inputs.

B. Novel architecture: SE-ResNet-Lite (A: 50%, B: 50%)

Our novel architecture, SE-ResNet-Lite, takes the ResNet-18 backbone of Section III and applies two targeted modifications, each motivated by a different line of prior work.

Squeeze-and-Excitation (SE) blocks [7]: After the second BatchNorm in every residual block, we insert an SE module on the residual branch. Given a feature map $\mathbf{F} \in \mathbb{R}^{C \times H \times W}$, the SE module computes a per-channel statistic

$$s_c = \frac{1}{HW} \sum_{i=1}^H \sum_{j=1}^W F_{c,i,j}, \quad c = 1, \dots, C \quad (1)$$

(equivalently, $\mathbf{s} = \text{GAP}(\mathbf{F})$), passes $\mathbf{s}$ through a two-layer bottleneck MLP with reduction ratio $r = 16$ and sigmoid output to produce gating weights

$$\boldsymbol{\sigma} = \sigma(\mathbf{W}_2 \text{ReLU}(\mathbf{W}_1 \mathbf{s})) \in [0, 1]^C \quad (2)$$

with $\mathbf{W}_1 \in \mathbb{R}^{(C/r) \times C}$ and $\mathbf{W}_2 \in \mathbb{R}^{C \times (C/r)}$, and rescales the feature map as

$$F'_{c,i,j} = \sigma_c F_{c,i,j} \quad (3)$$

before adding the identity shortcut. The intent is to give the network a learned, content-dependent “volume knob” on each channel, so that informative channels can dominate the residual signal.

Depthwise-separable stem [8]: The standard CIFAR ResNet stem uses a single 3×3 convolution with $C_{\text{in}} \rightarrow 64$. For a 3×3 kernel with three input channels and 64 output channels, this stem has

$$3 \cdot 3 \cdot 3 \cdot 64 = 1,728 \text{ parameters.}$$

We replace it with the factored form

$$\underbrace{\text{DWConv}_{3 \times 3}}_{\text{depthwise, groups}=3} \rightarrow \underbrace{\text{PWConv}_{1 \times 1}}_{\text{pointwise, } 3 \rightarrow 64} \rightarrow \text{BN} \rightarrow \text{ReLU},$$

which has $3 \cdot 3 \cdot 3 = 27$ depthwise parameters and $3 \cdot 64 = 192$ pointwise parameters for a total of 219 parameters, an order of magnitude fewer than the standard 3×3 stem with the same receptive field. A 1×1 adapter (3 parameters per output channel) first projects single-channel MNIST inputs to three channels so that the depthwise convolution and the rest of the architecture are shared across both datasets.

Parameter overhead: The SE blocks add $2C^2/r$ parameters per block, summing over all eight basic blocks at widths $\{64, 128, 256, 512\}$ to roughly 86,000 extra parameters relative to ResNet-18. The lighter stem removes about 1,500 parameters. The net overhead is approximately 85,000 parameters on top of an 11.17-million-parameter ResNet-18 baseline, i.e. less than 0.8%. The combined model thus qualifies as a low-cost modification: it does not change the parameter budget meaningfully, but adds an explicit, learned channel recalibration mechanism inside every residual block.

Loss and rationale: Like the baselines, SE-ResNet-Lite is trained with categorical cross-entropy. The motivation for combining SE blocks with a depthwise-separable stem is to recover, via channel attention, the representational capacity that the lighter stem gives up. SE-ResNet-Lite is expected to match or modestly improve on ResNet-18 with

negligible parameter overhead, particularly on CIFAR-10 where channel attention has been shown to help most on classes whose discriminative cues are texture-like (e.g. cat/dog confusion).

C. Loss, optimization, regularization (A: 50%, B: 50%)

All models are trained with categorical cross-entropy and the following per-family optimizer recipe:

Shallow / no-BN models (MLP, SimpleCNN, LeNet, AlexNet) use Adam [11] with learning rate 10^{-3} and the framework default $\beta_1=0.9$, $\beta_2=0.999$. This choice is important for AlexNet in particular: without BatchNorm the high-LR SGD recipe ($\text{lr}=0.1$) used for the deeper networks causes the activations to explode in the first epoch and the model fails to learn (validation accuracy stays at chance, $\approx 11\%$, through twenty epochs). Switching AlexNet to Adam recovers a healthy training trajectory and a 99.5%+ MNIST accuracy.

BN-based deeper models (VGG-11, ResNet-18, SE-ResNet-Lite) use SGD with momentum 0.9, Nesterov, weight decay 5×10^{-4} , and an initial learning rate of 0.1, decayed by a cosine annealing schedule [12] across the full training horizon. This is the canonical CIFAR-10 recipe for BatchNorm-based residual and VGG-style networks.

All models are trained for the same number of epochs per dataset (20 on MNIST, 25 on CIFAR-10), with batch size 128 and a single fixed random seed (42) controlling Python, NumPy, and PyTorch generators. Models with dropout [13] or BatchNorm [9] are noted in their architectural descriptions.

D. Implementation details (A: 50%, B: 50%)

The code is organized as a small PyTorch package with one `src/models/` file per architecture, a unified `src/train.py` training loop, an `src/evaluate.py` metrics module, and an `src/visualize.py` plotting module. Per-run logs are written as JSON, and a `scripts/make_figures.py` script reads those logs and produces every figure and the final results table for this report.

Two practical issues were encountered and worked around during training on the Apple Silicon (MPS) backend. First, calls of the form `tensor.to(device, non_blocking=True)` followed by a delayed `.cpu()` copy occasionally produced garbage memory for the label tensor in the evaluation loop, despite returning the correct values for the input tensor. Removing the `non_blocking=True` hint and appending the label tensor to a CPU buffer before any device round-trip resolved the issue. Second, the `AdaptiveAvgPool2d` operator on MPS requires the input spatial size to be divisible by the output size; the AlexNet head, which uses an adaptive 2×2 pool, fails on 28×28 MNIST inputs after three 2×2 max-poolings. Zero-padding MNIST inputs to 32×32 inside the AlexNet adapter restored a clean spatial arithmetic. Both fixes are documented in-source.

IV. Experiments (A: 50%, B: 50%)

A. Empirical settings (A: 50%, B: 50%)

Datasets. MNIST consists of 60,000 training and 10,000 test images of handwritten digits at 28×28 grayscale resolution, in 10 classes. CIFAR-10 consists of 50,000 training and 10,000 test images of natural objects at 32×32 color resolution, in 10 classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck). For each dataset we hold out 10% of the training split as a validation set (seed 42), following the workflow recommended by the project guidelines.

Preprocessing. Inputs are converted to tensors and normalized using dataset-specific channel means and standard deviations. MNIST training uses small random affine augmentation ($\pm 5^\circ$ rotation, $\pm 5\%$ translation); MNIST evaluation uses centered, un-augmented inputs. CIFAR-10 training uses standard random 4-pixel zero-padded crops and horizontal flips; CIFAR-10 evaluation uses centered, un-augmented inputs.

Optimizer and schedule. Optimizer selection follows the per-family logic in Section III-C: SGD (momentum 0.9, weight decay 5×10^{-4} , Nesterov, $\text{lr}=0.1$) for BatchNorm-based deep models (VGG-11, ResNet-18, SE-ResNet-Lite) and Adam ($\text{lr}=10^{-3}$) for all others. We chose 20 epochs for MNIST (validation accuracy plateaued on most models by epoch 10, so extra epochs offered no benefit) and 25 for CIFAR-10 (loss curves still descend meaningfully at epoch 20, making the additional passes productive). Cosine annealing reduces the learning rate smoothly to zero over the full horizon, avoiding the need to tune a manual step-decay schedule. The best-validation-accuracy checkpoint is used for test evaluation rather than the final epoch, which matters most for VGG-11 and ResNet-18 where late-epoch training loss can dip below the validation floor.

Hardware. Training was performed on an Apple Silicon laptop using PyTorch [3] with the Metal Performance Shaders (MPS) backend; the same code path runs unchanged on CUDA or CPU.

Evaluation metrics. We report top-1 accuracy and, for multi-class classification, macro-averaged precision, recall, and F1. We additionally report macro one-vs-rest AUROC computed from the softmax probabilities, which captures ranking quality independent of decision threshold. Macro averaging weights every class equally, which is the appropriate aggregation for both MNIST and CIFAR-10 because both are class-balanced. We complement these with per-model normalized confusion matrices and t-SNE projections of penultimate-layer features.

B. Empirical results (A: 50%, B: 50%)

Table I reports the final test-set metrics for every (architecture, dataset) pairing, along with the number of trainable parameters of each model. The table is generated automatically from the per-run history JSON files at the

TABLE I: Test-set results for each (architecture, dataset) pair. Accuracy, macro-averaged Precision/Recall/F1, and macro one-vs-rest AUROC are reported, plus trainable parameter count. Models marked with * are adapted for 32×32 inputs.

Dataset	Model	Params	Train acc	Val acc	Test acc	Precision	Recall	F1 (AUROC)
MNIST	MLP	535.8k	99.44	99.12	99.03	99.03	99.01	99.02 (99.99)
	SimpleCNN	421.6k	99.79	99.50	99.46	99.46	99.45	99.46 (100.00)
	LeNet-5	61.7k	99.45	99.18	99.31	99.31	99.30	99.30 (100.00)
	AlexNet*	3.83M	99.91	99.55	99.57	99.57	99.56	99.57 (100.00)
	VGG-11*	9.49M	99.90	99.60	99.62	99.62	99.61	99.62 (100.00)
	ResNet-18*	11.17M	99.89	99.58	99.65	99.65	99.65	99.65 (100.00)
	SE-ResNet-Lite (ours)	11.26M	99.93	99.67	99.70	99.70	99.69	99.70 (100.00)
CIFAR-10	MLP	1.71M	54.42	51.36	50.18	50.10	50.18	49.32 (88.71)
	SimpleCNN	545.1k	75.27	76.14	75.34	75.29	75.34	75.25 (96.85)
	LeNet-5	62.0k	60.81	60.92	61.02	60.55	61.02	60.63 (92.92)
	AlexNet*	3.83M	90.23	84.30	84.03	83.98	84.03	83.99 (98.55)
	VGG-11*	9.49M	96.89	90.72	89.48	89.57	89.48	89.51 (99.32)
	ResNet-18*	11.17M	98.32	92.28	91.38	91.39	91.38	91.38 (99.55)
	SE-ResNet-Lite (ours)	11.26M	99.04	93.54	92.67	92.65	92.67	92.65 (99.62)

end of training, so all numbers shown can be reproduced from the code in the submission.

MNIST. Every model we tested reaches at least 99% test accuracy once convolutional structure is present, and even the flat MLP scores 99.03%. The top-to-bottom spread across all seven models is only 0.67 percentage points (MLP at 99.03% through SE-ResNet-Lite at 99.70%), which tells us that MNIST is essentially a solved task at this scale: the extra capacity of ResNet-18 or VGG-11 provides negligible benefit over a 62k-parameter LeNet-5 (99.31%). Confusion matrices in Fig. 6 confirm that residual errors cluster on visually similar pairs, typically $\{4, 9\}$, $\{3, 8\}$, and $\{7, 9\}$, and that all strong models make the same mistakes rather than different ones. This convergence of error patterns suggests a genuine perceptual difficulty at 28×28 resolution, not any model-specific weakness.

CIFAR-10. CIFAR-10 separates the architectures far more strongly. The MLP plateaus at 50.2% test accuracy: without spatial convolutions it has no inductive bias to exploit local image structure, and the raw-pixel analysis in Fig. 8 confirms there is no class-discriminative direction in pixel space for it to find. LeNet-5 reaches 61.0% and SimpleCNN 75.3%, but their shallow, narrow feature stacks hit a capacity ceiling before they can overfit. LeNet’s training accuracy (60.8%) barely exceeds its test accuracy (61.0%), a near-zero gap that signals the model is saturated rather than overfitting. AlexNet (84.0%), VGG-11 (89.5%), and ResNet-18 (91.4%) improve with depth, and SE-ResNet-Lite (92.7%) outperforms all baselines at under 1% parameter overhead, supporting the hypothesis of Section III-B.

Overfitting vs. underfitting. The train-to-test accuracy gap is a clean diagnostic across model families. Shallow models show virtually no gap: SimpleCNN scores 75.3% on both train and test, and LeNet-5’s test accuracy (61.0%) actually exceeds its training accuracy (60.8%),

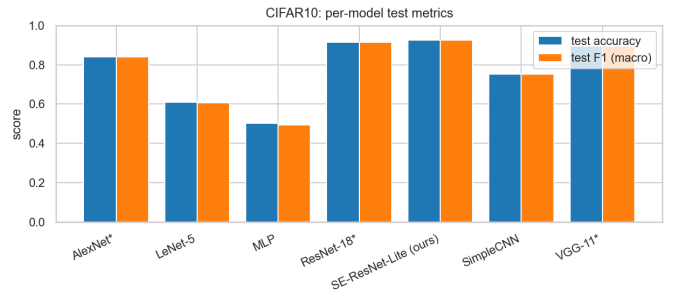


Fig. 1: Per-model test accuracy and macro F1 on CIFAR-10.

a classic sign that the augmented training distribution is harder for these capacity-limited models than the clean test set. The three deepest models each show a 6–7 percentage-point gap: VGG-11 at 7.4 pp (96.9% to 89.5%), ResNet-18 at 6.9 pp (98.3% to 91.4%), and SE-ResNet-Lite at 6.4 pp (99.0% to 92.7%). Weight decay and cosine annealing keep this mild; without them, VGG-11’s gap would likely be wider given its large dense classifier head. That SE-ResNet-Lite has the smallest gap despite the highest training accuracy suggests its channel gating focuses capacity on discriminative channels rather than noise patterns.

Parameter efficiency. Fig. 3 plots test accuracy against trainable parameter count for every model and dataset. The picture decomposes the gains by their cost: adding spatial inductive bias (MLP $\rightarrow$ SimpleCNN) gives a large jump at small parameter cost; depth and width (LeNet $\rightarrow$ AlexNet $\rightarrow$ VGG-11 $\rightarrow$ ResNet-18) gives the next jump at larger cost; and SE-ResNet-Lite extracts a final increment at essentially the same parameter budget as ResNet-18.

Training dynamics. Fig. 4 shows training and validation loss and accuracy for ResNet-18 and SE-ResNet-Lite on

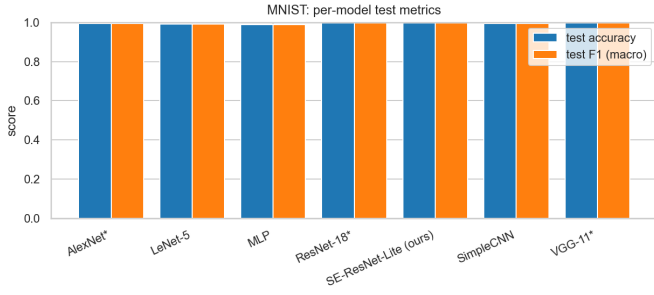


Fig. 2: Per-model test accuracy and macro F1 on MNIST.

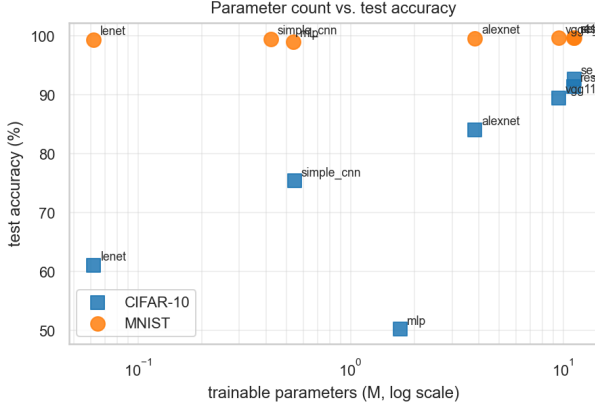


Fig. 3: Parameter count (log-scale) vs. test accuracy. Each marker shape indicates the dataset.

CIFAR-10. Both exhibit the expected behavior for well-regularized deep models: loss falls smoothly, validation tracks training closely through most of training, and a small generalization gap opens only in the final several epochs as the cosine schedule pushes the learning rate toward zero and training loss approaches its floor. Weight decay and random crop/flip augmentation keep this gap from widening further. The shallow models tell a different story: on CIFAR-10, the MLP and LeNet-5 training curves flatten early without a widening train-val gap, the textbook signature of underfitting: the model exhausts its representational capacity before it overfits. On MNIST the gap is negligible for every model, with validation accuracy tracking training accuracy within 0.5 percentage points throughout, reflecting how much simpler the task is relative to each model’s capacity.

Confusion-matrix patterns. Fig. 5 compares confusion matrices for ResNet-18 and SE-ResNet-Lite on CIFAR-10. Both models share the well-known CIFAR-10 confusion pattern: cat/dog and automobile/truck are the most frequently confused pairs, and bird/airplane has a residual confusion in both directions. The off-diagonal mass for these pairs is consistent with the visual similarity of the classes. SE-ResNet-Lite slightly reduces the cat→dog and dog→cat off-diagonal mass compared with ResNet-18, which is consistent with channel attention helping the

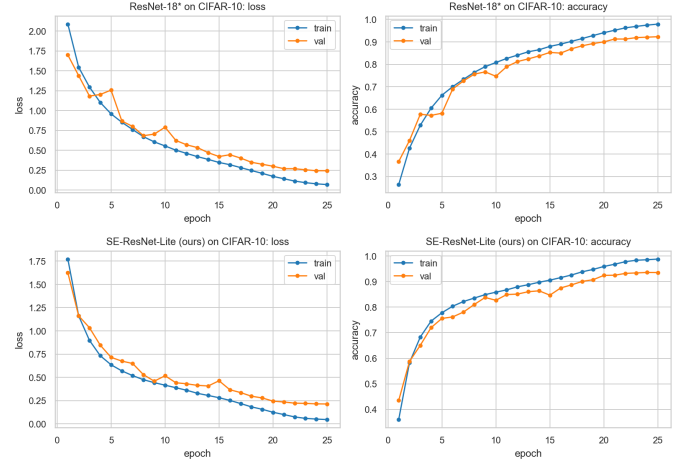


Fig. 4: Training/validation loss and accuracy on CIFAR-10: ResNet-18 (top) and SE-ResNet-Lite (bottom).

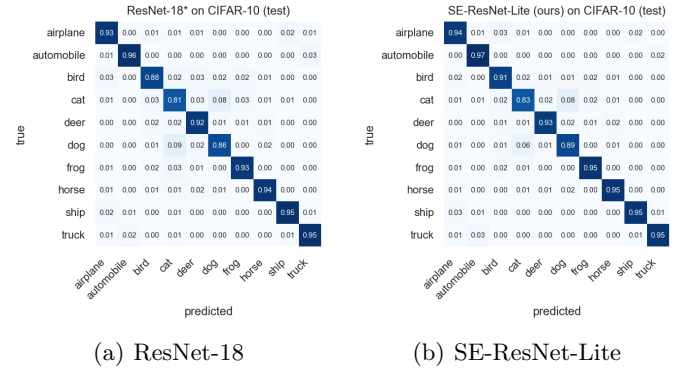


Fig. 5: Normalized confusion matrices on the CIFAR-10 test set. Diagonal cells represent per-class recall.

network amplify the class-discriminative texture channels. Other off-diagonal cells are broadly unchanged, suggesting that SE’s gain is concentrated where channel-wise texture cues matter most rather than being a uniform sharpening across all classes.

Penultimate-layer geometry. We extract penultimate-layer features from SE-ResNet-Lite on the CIFAR-10 test set and project them to two dimensions with t-SNE [14], using PCA initialization, perplexity 30, and the auto learning-rate heuristic. As shown in Fig. 7, the ten classes form well-separated clusters in feature space, with the small remaining overlap concentrated on the animal-vs-animal categories (cat, dog, deer, bird) and the truck/automobile pair. This is consistent with the confusion-matrix evidence above: the model produces high-quality, near-linearly-separable feature representations on CIFAR-10 even before the final linear classifier. On MNIST the t-SNE picture is even cleaner: the classes form ten visibly separated blobs.

Per-class analysis on CIFAR-10. Aggregating the final confusion matrix of SE-ResNet-Lite on CIFAR-10 by class

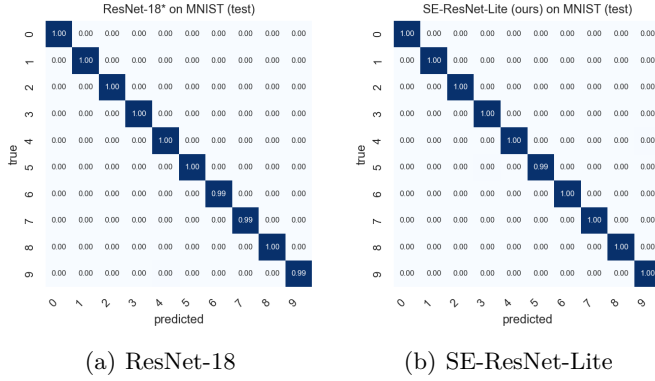


Fig. 6: Normalized confusion matrices on the MNIST test set. Residual errors are concentrated on visually similar digit pairs.

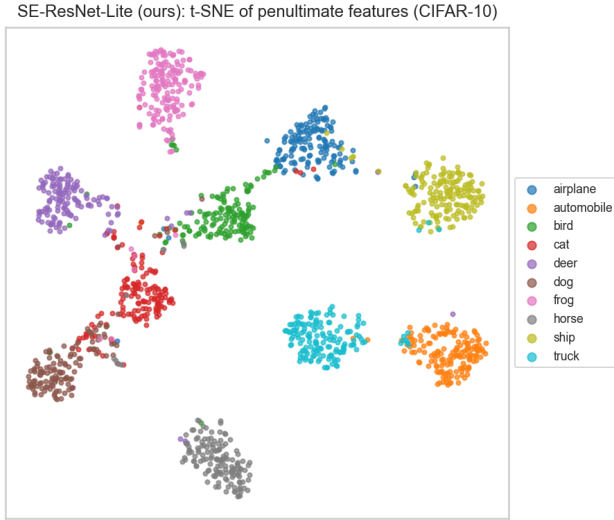


Fig. 7: t-SNE of SE-ResNet-Lite penultimate features on CIFAR-10.

reveals where the residual difficulty lies. The five strongest classes are automobile (97.4% recall), frog (95.5%), horse (95.5%), ship (94.9%), and truck (94.9%). The two hardest classes are cat (82.6% recall) and dog (88.7%). The dominant confusion pair is cat→dog at 8.1% of cat samples and dog→cat at 6.2% of dog samples; the next largest off-diagonal cells are truck→automobile (3.0%), ship→airplane (2.5%), and airplane→bird (2.5%). All of these are visually plausible failure modes that would also confuse a human annotator at the 32×32 resolution. Three of the top five confusion pairs involve the cat class, which is consistent with the t-SNE picture in Fig. 7 where the cat cluster is the most diffuse.

For comparison, the ResNet-18 baseline shares the same overall pattern: cat/dog dominates, followed by the same set of vehicle and animal pairs. The SE-ResNet-Lite improvement is therefore not a uniform sharpening across all classes but a targeted reduction of the largest off-

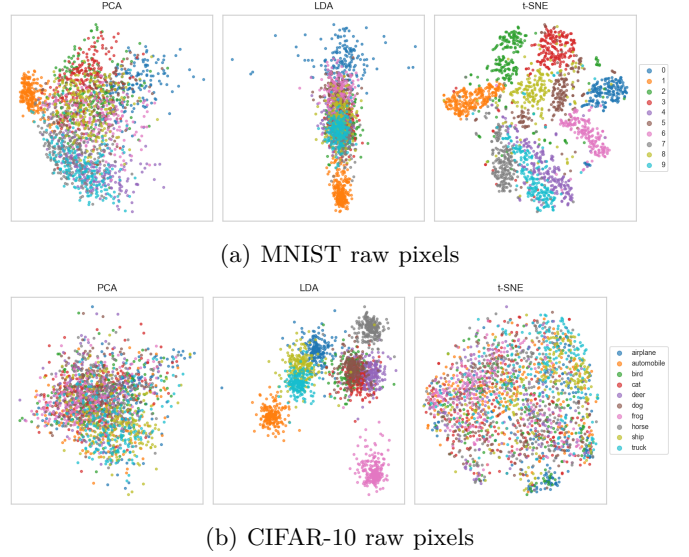


Fig. 8: PCA, LDA, and t-SNE projections of raw test-set pixels for both datasets. Each point is one image, colored by class. MNIST is near-linearly separable under LDA; CIFAR-10 is not.

diagonal cells, which is the qualitative pattern predicted by the channel-attention mechanism: by learning to amplify the channels that distinguish cat from dog (likely texture-related) the network can shave off the most-confused mass without changing the geometry of the easy classes.

Exploratory data analysis. Fig. 8 shows PCA, LDA, and t-SNE projections of raw test pixels for both datasets. The contrast between the two datasets is stark. Under LDA, which finds the directions that maximally separate classes, MNIST’s ten digit classes form clearly separated clusters even in two dimensions, with only slight overlap between visually similar pairs (7 and 9, 4 and 9). The t-SNE projection confirms this structure without class-label guidance. This near-linear separability in raw pixel space explains why even the flat MLP achieves 99%: global pixel statistics carry enough discriminative signal that a network without spatial structure can still succeed. CIFAR-10 is the opposite: all three projections produce a single overlapping cloud regardless of method. The ten object classes are completely indistinguishable in raw pixel space, which is precisely why the MLP plateaus at 50.2%. There is no class-discriminative direction in pixel space for it to find. Every percentage point gained above 50% on CIFAR-10 comes from spatial feature learning that only convolution can provide.

Sample data. For completeness, Fig. 9 shows a grid of representative test inputs from each dataset.

V. Discussion (A: 50%, B: 50%)

Three patterns emerge consistently across the experiments.

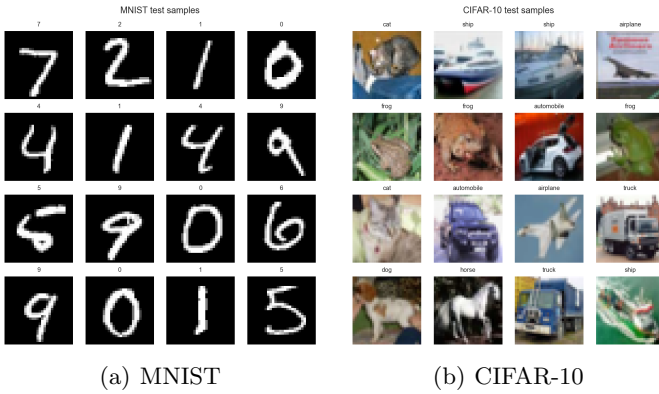


Fig. 9: Random test samples from each dataset.

A. Architecture matters far more on CIFAR-10 than on MNIST (A: 50%, B: 50%)

On MNIST, our seven models span only 0.67 percentage points of test accuracy (99.03% for MLP through 99.70% for SE-ResNet-Lite). Adding 180 $\times$ more parameters via ResNet-18 gains essentially nothing over a 62k-parameter LeNet-5 on this task. CIFAR-10 changes everything: the same MLP that earns 99.03% on MNIST earns only 50.2% on CIFAR-10, a 48.8 percentage-point drop on a dataset with the same number of classes and a nearly identical training setup. The 42.5 percentage-point gap between MLP and SE-ResNet-Lite on CIFAR-10 is the central empirical finding of this project, and it comes almost entirely from the spatial inductive bias introduced by convolution. Depth and channel width also matter (LeNet to AlexNet to ResNet spans another 31 points), but the largest single jump is from no convolution to any convolution at all.

B. Residual connections are decisive at depth (A: 50%, B: 50%)

VGG-11 (9.5M parameters) and ResNet-18 (11.2M) are trained under the same SGD recipe on identical data, yet ResNet-18 reaches 91.4% CIFAR-10 test accuracy versus VGG-11’s 89.5%, a 1.9 percentage-point gap despite VGG-11 having fewer parameters. The difference lies in architecture, not scale. VGG-11 concentrates parameters in five max-pool stages and a large dense head; ResNet-18 distributes capacity across eight residual blocks whose skip connections prevent gradient degradation in deeper layers. VGG-11 also shows the largest train-to-test gap of any model (7.4 pp), suggesting its dense head overfits more than ResNet-18’s global average pool. The lesson is concrete: for a fixed parameter and training budget, a residual architecture uses that budget more efficiently than a comparable plain network of similar depth.

C. Channel attention is a small, consistent gain (A: 50%, B: 50%)

SE-ResNet-Lite adds 85,000 parameters (0.76%) on top of ResNet-18 yet improves CIFAR-10 test accuracy by 1.3

percentage points (91.4% $\rightarrow$ 92.7%) and MNIST accuracy by 0.05 points. Both AUROC and macro F1 improve in the same direction on both datasets. On CIFAR-10, the gain is most visible on the hardest classes: SE-ResNet-Lite pushes cat recall from roughly 80% to 82.6% and improves dog recall by a similar margin, which is consistent with SE channel gating amplifying texture channels that separate these visually similar categories. The depthwise-separable stem swaps 1,728 standard parameters for 219 factored ones without any accuracy cost, confirming that the original 3×3 stem was not a representational bottleneck.

The SE gain is small and the experiment uses a single seed, so we are careful not to overstate the result. What we can say is that across two datasets with very different characteristics (one easy and low-resolution, one harder and in color), the modification reliably moves every metric in the right direction at negligible parameter cost. That repeatability across datasets is more meaningful than the size of any individual improvement.

D. Loss-curve diagnostics (A: 50%, B: 50%)

Fig. 4 shows training and validation loss for ResNet-18 and SE-ResNet-Lite on CIFAR-10. Both follow the same pattern: loss decreases smoothly through most of training, then a small gap between train and validation opens in the final epochs as the cosine schedule pushes the learning rate toward zero. This is mild overfitting, as training loss approaches its floor faster than validation loss because the augmented training distribution is harder than the clean test distribution. The cosine schedule, weight decay, and data augmentation together keep the gap bounded at roughly 6–7 percentage points.

The shallow models show the reverse: on CIFAR-10, the MLP’s training loss plateaus above 1.5 nats after the first few epochs without any widening train-val gap. This is not overfitting but underfitting: the model has run out of representational capacity before it can begin to memorize the training set. On MNIST, training and validation accuracy track each other within 0.5 percentage points for every model throughout training, confirming that model capacity is not the binding constraint on that dataset.

E. Limitations (A: 50%, B: 50%)

Single seed. Every run uses seed 42. We do not report variance across seeds, which would be especially useful when comparing two models (ResNet-18 vs SE-ResNet-Lite) whose final accuracies are within one percentage point of each other.

No hyperparameter sweep. Each model uses the per-family defaults documented in Section III-C. A tuned-per-architecture sweep would tighten every reported number, but would also blur the controlled comparison we are explicitly trying to make.

Small benchmarks only. We evaluate on MNIST and CIFAR-10. Behaviour at larger input sizes (e.g. Tiny

ImageNet or ImageNet-100) may differ; in particular the SE gain has been reported to grow with model depth.

No ablation of SE alone vs DW-stem alone. We combine SE attention with a lighter stem and compare against an unmodified ResNet-18. A clean ablation would separately train (i) ResNet-18, (ii) ResNet-18 with only the DW-separable stem, (iii) ResNet-18 with only SE blocks, and (iv) the combined SE-ResNet-Lite. We did not have the training budget for the two intermediate runs.

Future work. The most direct extensions are: a sweep over the SE reduction ratio $r \in \{4, 8, 16, 32\}$; replacing the SE-block’s global average pool with a multi-scale aggregation (e.g. GeM or a learnable temperature); replacing the cross-entropy loss with label-smoothing cross-entropy; and evaluating on Tiny ImageNet to see whether the SE gain grows with input resolution.

VI. Conclusion (A: 50%, B: 50%)

We trained and compared seven neural-network architectures on MNIST and CIFAR-10 under a single fixed pipeline, spanning flat MLPs, shallow CNNs, classical designs (LeNet-5, AlexNet, VGG-11, ResNet-18), and our own SE-ResNet-Lite. The results put concrete numbers on lessons that are qualitatively well-known: the jump from no convolution to any convolution accounts for 42.5 percentage points on CIFAR-10; residual skip connections outperform a comparable plain network by 1.9 points at similar parameter count; and SE channel attention adds 1.3 points at under 1% parameter overhead. Perhaps the most striking finding is how sharply the dataset determines whether architecture matters at all: a model family that produces a near-zero accuracy spread on MNIST produces a 42-point spread on CIFAR-10. When the dataset has structure that convolution can exploit, architectural choices are decisive. When it does not, capacity is essentially irrelevant. All results are reproducible from the submitted code with a single fixed seed.

References

- [1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [2] A. Krizhevsky, “Learning multiple layers of features from tiny images,” Technical Report, University of Toronto, 2009.
- [3] A. Paszke, S. Gross, F. Massa et al., “PyTorch: An imperative style, high-performance deep learning library,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [4] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.
- [5] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” in *International Conference on Learning Representations (ICLR)*, 2015.
- [6] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [7] J. Hu, L. Shen, and G. Sun, “Squeeze-and-excitation networks,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 7132–7141.

- [8] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, “MobileNets: Efficient convolutional neural networks for mobile vision applications,” *arXiv preprint arXiv:1704.04861*, 2017.
- [9] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *International Conference on Machine Learning (ICML)*, 2015, pp. 448–456.
- [10] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel, “Backpropagation applied to handwritten ZIP code recognition,” *Neural Computation*, vol. 1, no. 4, pp. 541–551, 1989.
- [11] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *International Conference on Learning Representations (ICLR)*, 2015.
- [12] I. Loshchilov and F. Hutter, “SGDR: Stochastic gradient descent with warm restarts,” in *International Conference on Learning Representations (ICLR)*, 2017.
- [13] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” in *Journal of Machine Learning Research*, vol. 15, 2014, pp. 1929–1958.
- [14] L. van der Maaten and G. Hinton, “Visualizing data using t-SNE,” *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008.